MODERNIZING THE MAINFRAME

WITH AGENTIC AI

by A. Vijay Aditya · IIT (BHU) Varanasi,India · Open Mainframe Project Summer 2025 Mentorship

LFX Mentorship Showcase • OSS Korea 2026

Open Mainframe Project · Linux Foundation

THE PROBLEM

⏰ 3 AM Data Center

Half those batch jobs process empty files. Reports go unread. Engineers fix JCL errors instead of building new solutions.

Weeks for Simple Answers?

"Can we cut costs?" takes weeks — digging old docs, chasing the one expert from the '90s, praying tribal knowledge sur vived.

🔗 ModernisationBlindness

A single COBOL job may move money, trigger compliance checks, and update dozens of reports. Without a dependency map, any change is a gamble.

WHAT IS MAIA

MAIA (Mainframe Agentic Intelligence Assistant) is a six-layer agenticAI system that turns a mainframe estate into an intelligent, queryable platform.

- Natural language queries — ask in plain English
- Knowledge graph built from COBOL + JCL via z/OSMF
- Vector search over runbooks, ADRs, incident history
- 4 specialist AI agents for cost, performance, modernisation, operations
- LLM reasons over your specific mainframe data
- Progressive trust model — observe → recommend → automate
- React web app connected to z/OSMF REST API

SYSTEM ARCHITECTURE — SIX-LAYER PIPELINE



1 DATA INGESTION

Layer 0 — GettingDataIn

z/OSMF REST API

- Datasets API → COBOL source (PDS libraries)
- Jobs API → execution history from JES
- Systems API → LPAR names and status

File Upload

- PDFs, Markdown, DOCX → text extracted
- Chunked into 500-800 token pieces
- Embedded → vectors stored in ChromaDB

SMF Offload

- Type 30 → job accounting records
- Type 72 → LPAR MIPS per hour
- Loaded into SQLite time-series store

2 KNOWLEDGE FOUNDATION

Layer 1 — Three Stores

Neo4j Graph Database

- Programs, Jobs, LPARs, Datasets as nodes
- CALLS · EXECUTES · TRIGGERS · READS edges
- Impact query: 0.3 seconds for full estate

ChromaDB Vector Store

- Runbooks, ADRs, incidents as embeddings
- Semantic search — meaning not keywords
- Runs locally — no data leaves network

SQLite Time-Series

- Job MIPS, elapsed time, return codes
- LPAR CPU utilisation per hour
- R4HA peak prediction and cost analysis

3 AI INTELLIGENCE

Layers2-4 — Brain + Agents

LLM + MCP

- Reads mainframe context.json (estate map)
- Decides which tools to call — no keyword rules
- Answers grounded in real graph/SMF data

LangGraph Orchestrator

- StateGraph with Orchestrator + 4 agents
- Conditional routing per query intent
- LangSmith traces every tool call + hop

React + FastAPI

- 5-tab web app — Chat, LPAR, Impact, Recs
- POST /api/chat → streaming LLM response
- Approve / Reject recommendation buttons

FOUR SPECIALIST AGENTS — LAYER 3

Cost Intelligence Agent

- Reads SMF Type 72 LPAR MIPS data
- Identifies R4HA billing tier spikes
- Recommends job rescheduling
- Predicts peak before it happens

Performance & Impact Agent

- Bottleneck detection from SMF
- DB2 buffer pool optimisation
- Analyses job SLA vs actual runtime
- Capacity planning per LPAR

Modernisation Agent

- Scores programs on 4 dimensions
- Rehost / Refactor / Replace decision
- Risk-aware sequenced roadmap

Operational Control Agent

- Searches ChromaDB runbooks
- Automated incident escalation
- Detects recurring failure patterns

CONCLUSION & FUTURE

MAIA demonstrates that the three core problemsofmainframe modernisation—operational waste, knowledge opacity, and modernisation blindness — can be addressed through a layered agentic AI architecture grounded in real mainframe data.

By connecting to z/OSMF, parsing COBOL and JCL with tree-sitter, and reasoning over Neo4j + ChromaDB + SQLite with LLMs and MCP servers, the system answers in seconds what currently takes weeks.

GitHub Repo
prototype code

ACKNOWLEDGMENTS

Mentor: Suresh Tadishetty

Program: LFX Mentorship — Summer 2025

Project: Modernizing Mainframe with Agentic AI

Connect with me!

vijayaditya1977@gmail.com

linkedin.com/in/a-vijay-aditya/